

PREDICCIÓN

Aprendizaje Supervisado

Clasificación

Árboles Decisión

Random Forest

K-Nearest Neighbor

SVM

Naïve Bayes

Redes Neuronales

Regresión

Lineal

No-Lineal

Logística

DESCRIPCIÓN

Aprendizaje No-supervisado

Asociación

Reglas de Asoc

Apriori

Segmentación

K-Means, Jerárquicos

DBSCAN, HDBSCAN

Autoencoders

Restricted Boltzman
Machines (RBM)

ANÁLISIS

Semi-supervisado

COMPARACIÓN

Word Embedding

Cálculo Distancia

Transformers

K-means

K-medias

Algoritmo de aprendizaje no supervisado

**No requiere de etiqueta o clase asociada a la instancia que
guía a la creación del modelo**

**Objetivo: encontrar agrupamientos que se encuentran de una
manera natural dentro del conjunto de datos**

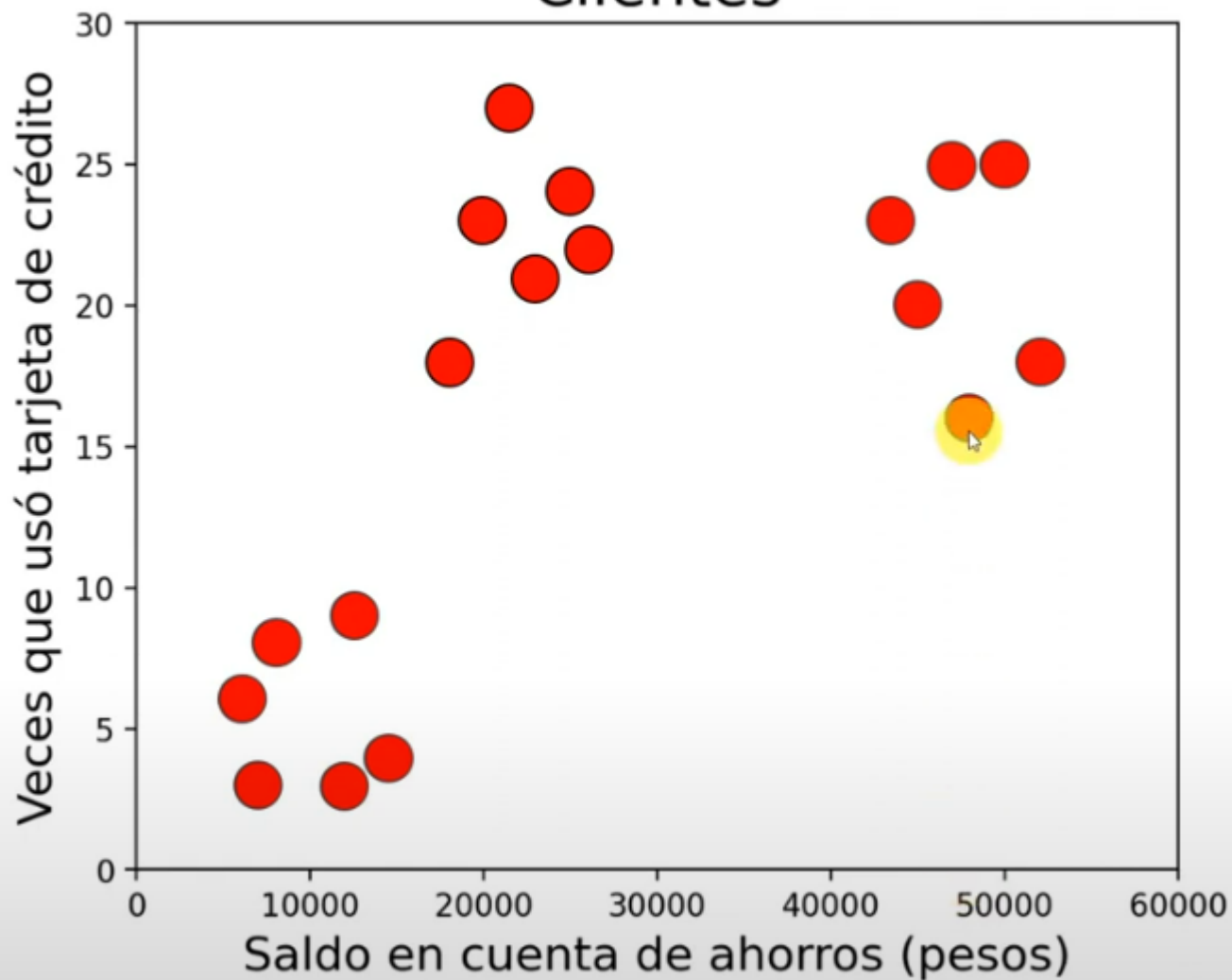


Contexto: Segmentación de clientes bancarios

Suponer que un Banco necesita saber si sus clientes se agrupan de forma natural utilizando dos características: saldo en la cuenta de ahorro y la segunda característica las veces que los clientes han utilizado la tarjeta de crédito en el último trimestre.

En el gráfico se observan los datos de 18 clientes.

Cientes

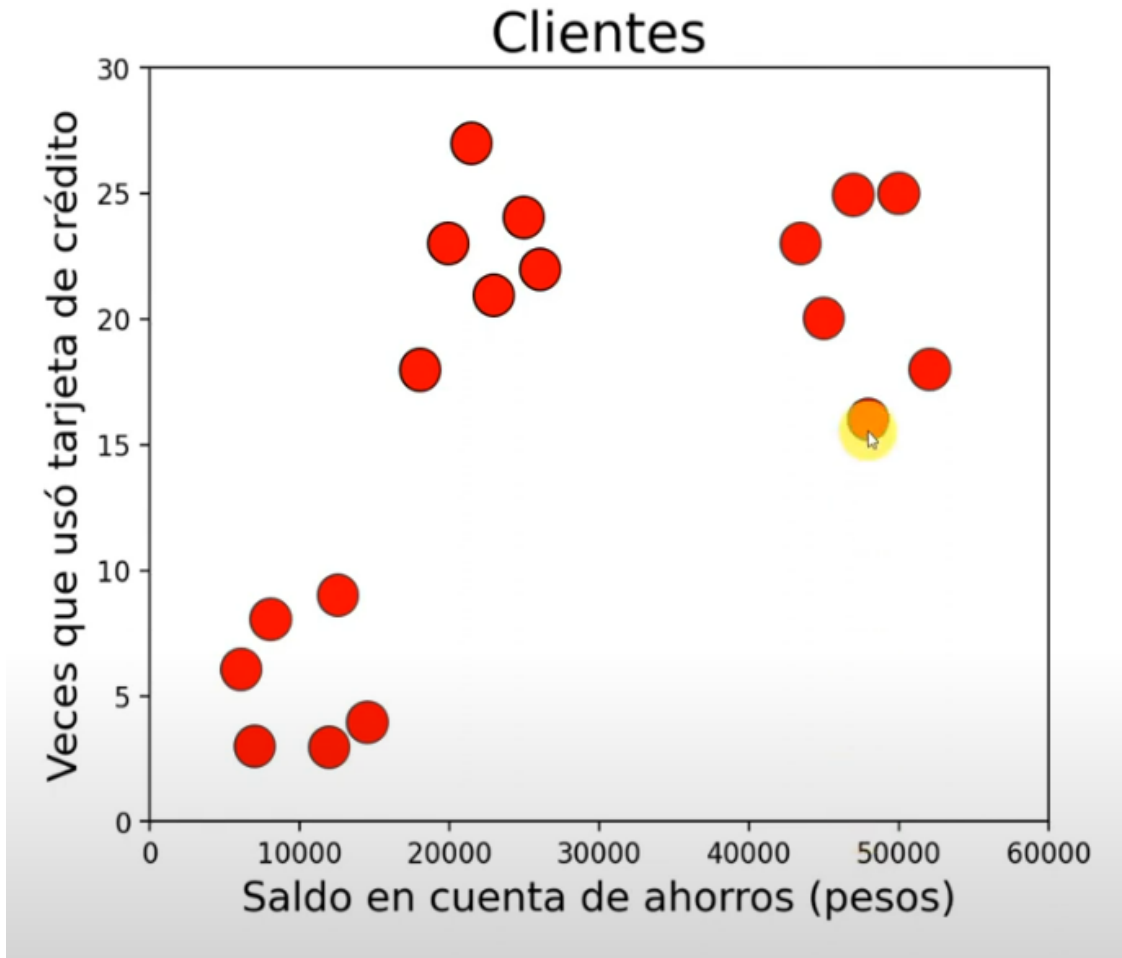


Cada punto representa un cliente.

En particular, vemos un cliente que tiene un saldo aproximado de 48.000 pesos en su cuenta de ahorro y que en el último trimestre ha usado aproximado de 16 veces la tarjeta de crédito.

K-means

K-medias



Este algoritmo permite la premisa de suponer que existe un determinado número o cantidad **K** de agrupamientos o clústers dentro de nuestros datos.

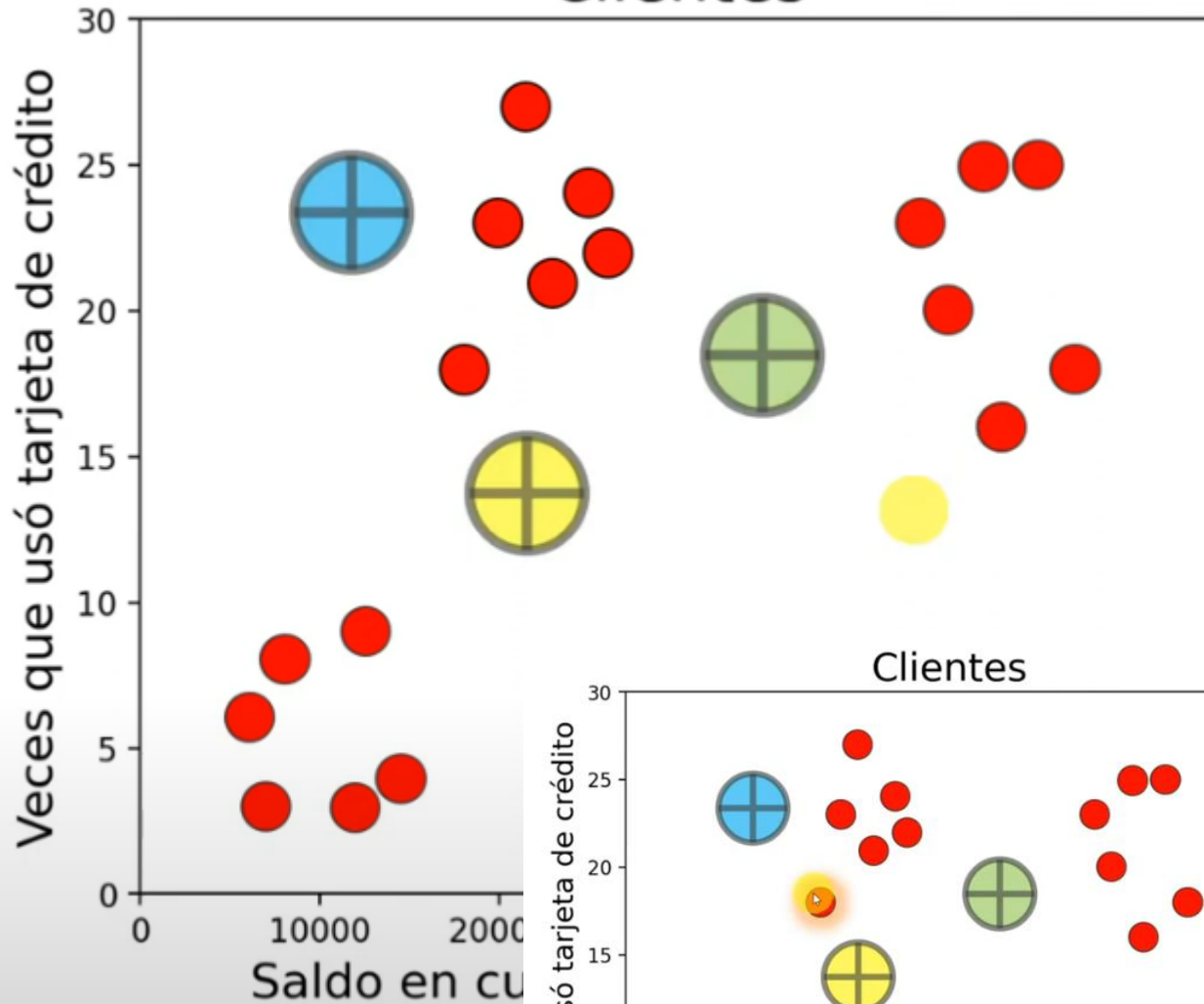
Podrían ser:

- K=2 cluster
- K=3 cluster
- K=4 cluster... y así sucesivamente

Todo depende para que se está utilizando el algoritmo. Podría que venga determinado por un requerimiento del negocio o a lo mejor el banco posee 3 diferentes productos que quisiera otorgarlo a 3 potenciales agrupamientos de clientes. O También existe algunos mecanismos utilizando métricas disasociadas del negocio que nos ayudan a determinar el número adecuado de clusters que podrían existir dentro de nuestros datos.

La K viene de una suposición que se realiza en el algoritmo.

Cientes



K-means

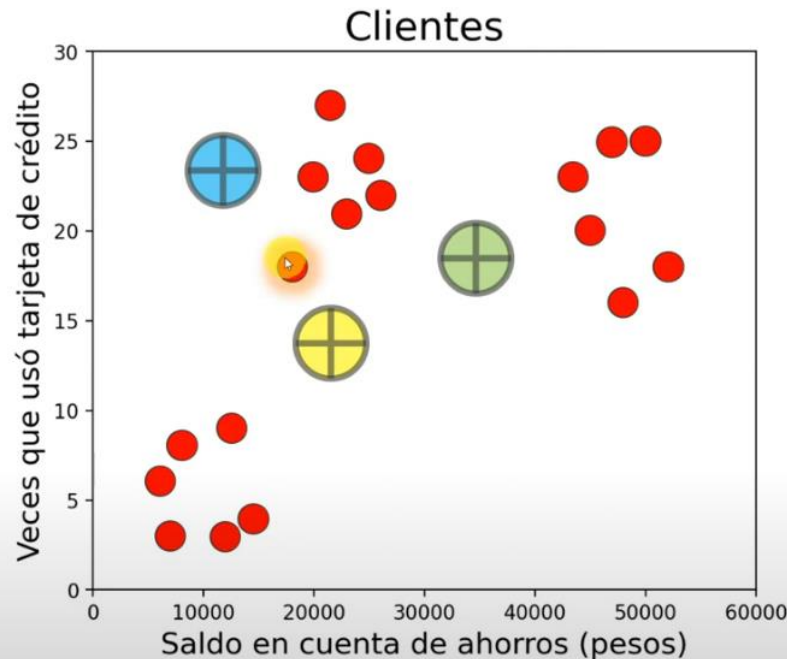
K-medias

1 paso:

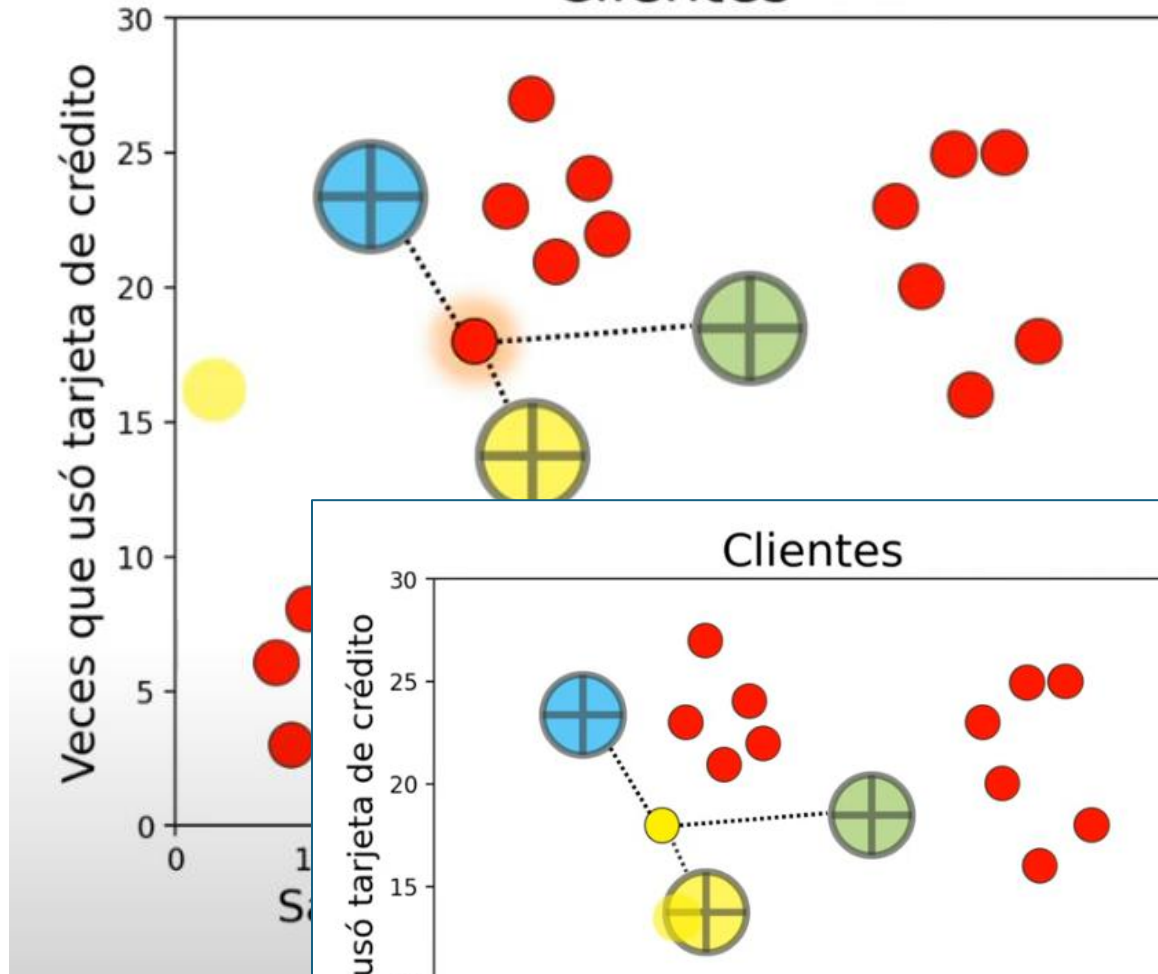
Suponer que tenemos $K=3$ clusters

Se crean 3 centroides que se ubican de forma aleatoria dentro de nuestro set de datos.

2 paso: Seleccionar un punto cualquiera del set de datos

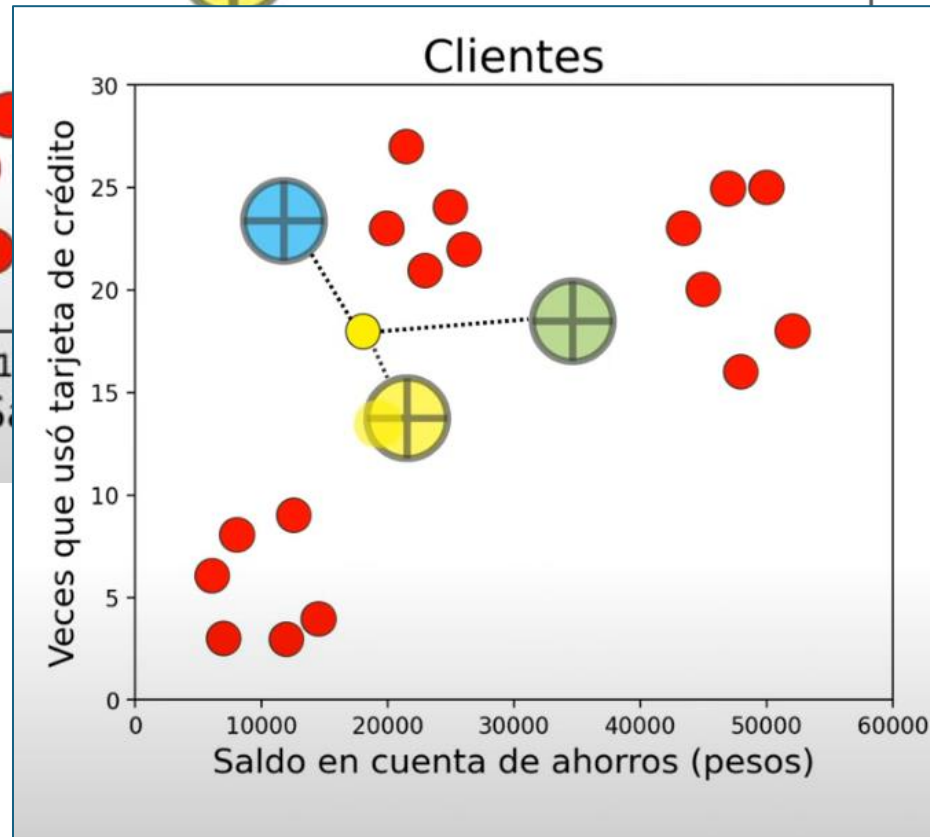


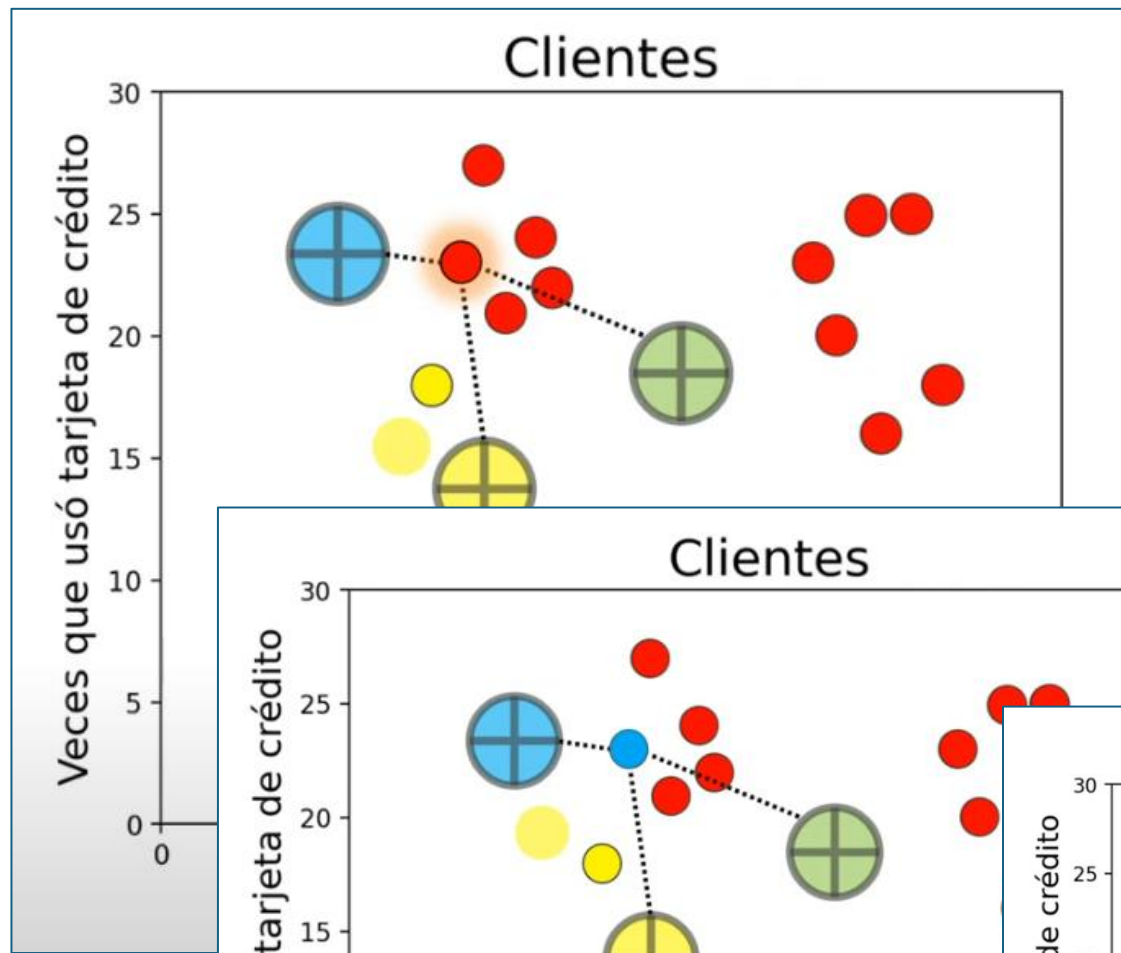
Cientes



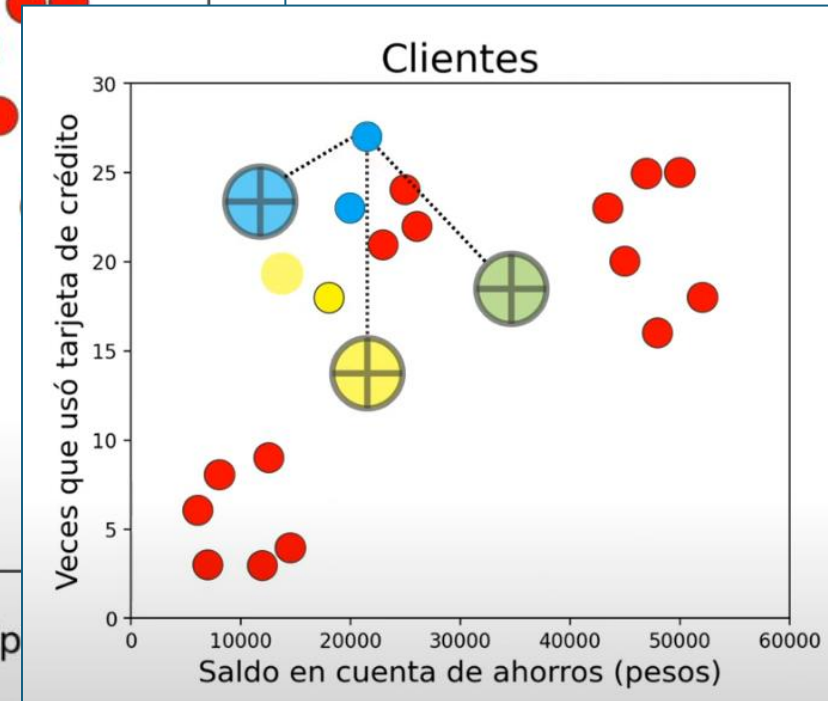
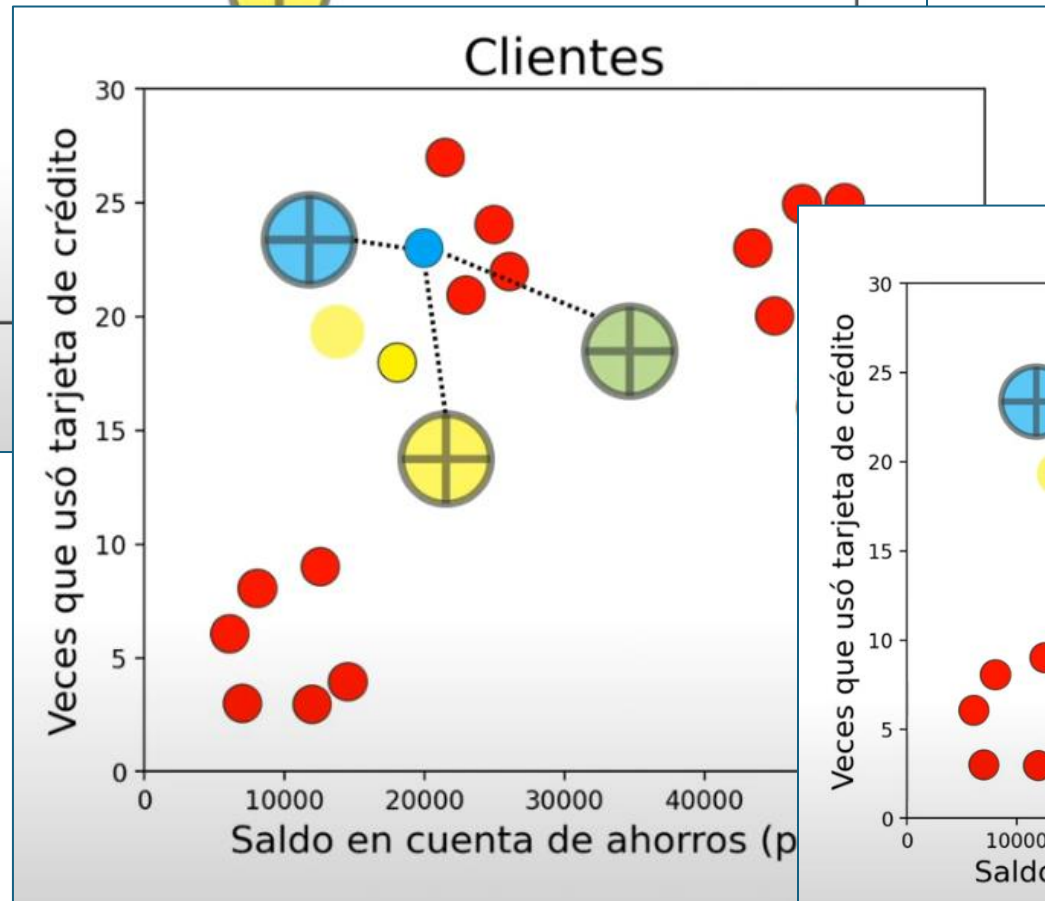
Paso 3: medir la distancia euclidiana de un punto aleatorio del set de datos a los centroides.

Visualmente, el centroide que está más cerca (menor distancia) al punto es el centroide amarillo por lo tanto al punto lo asignamos inicialmente al clúster amarillo.

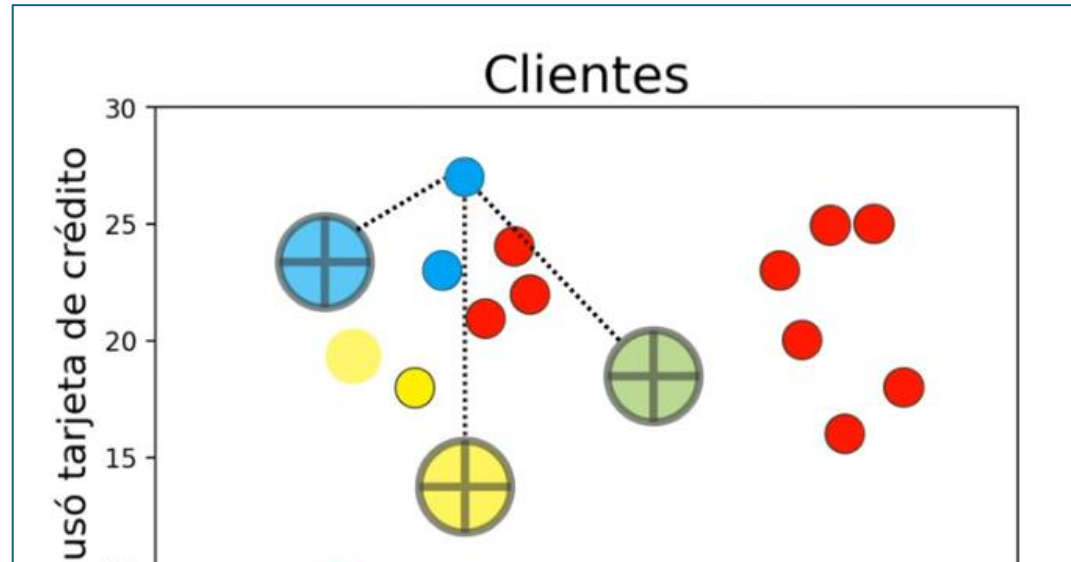




Repetimos el paso 3

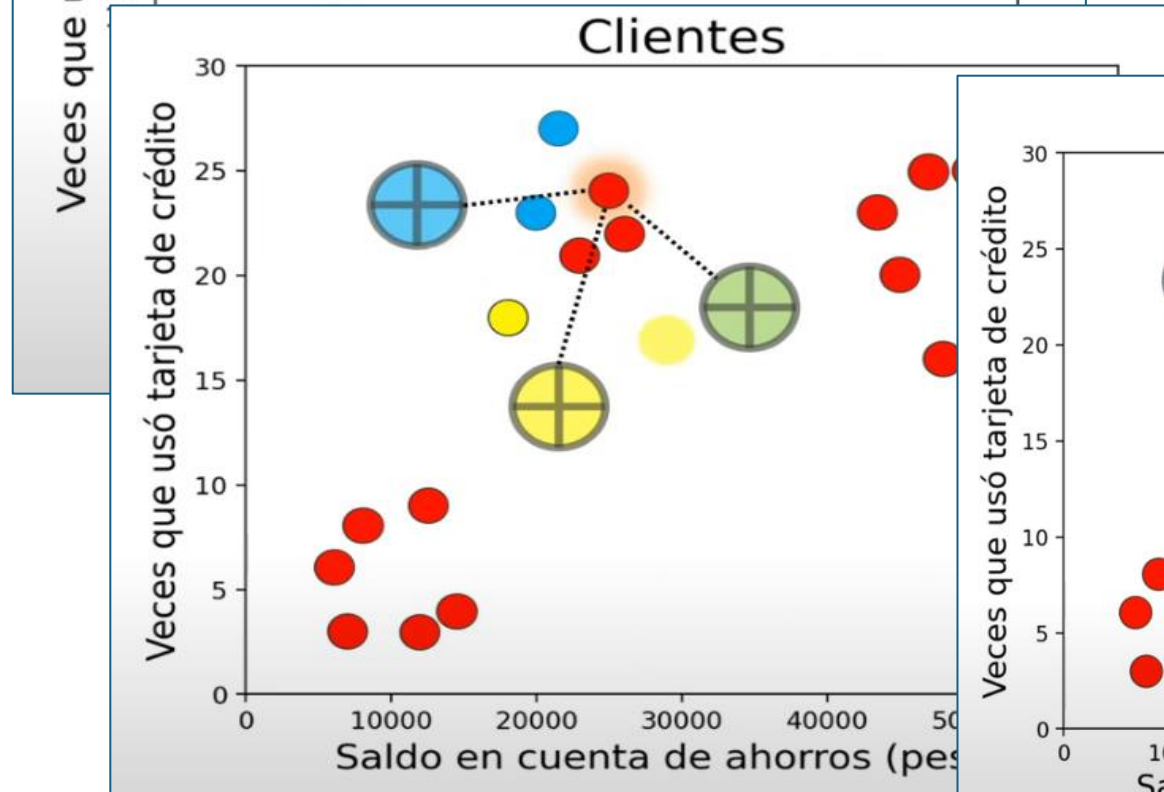


3

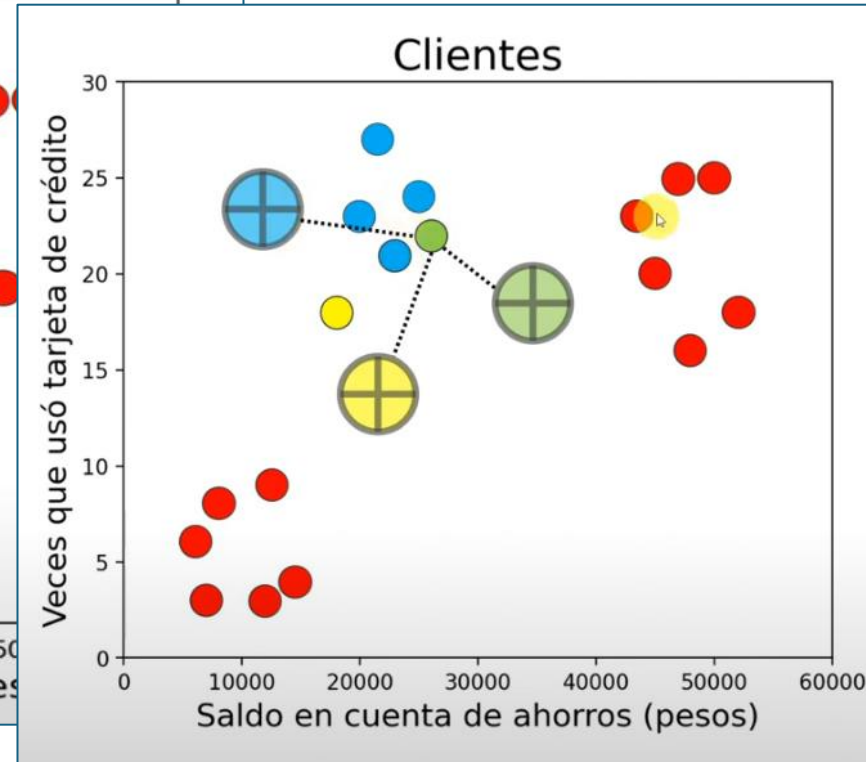


Repetimos el paso 3

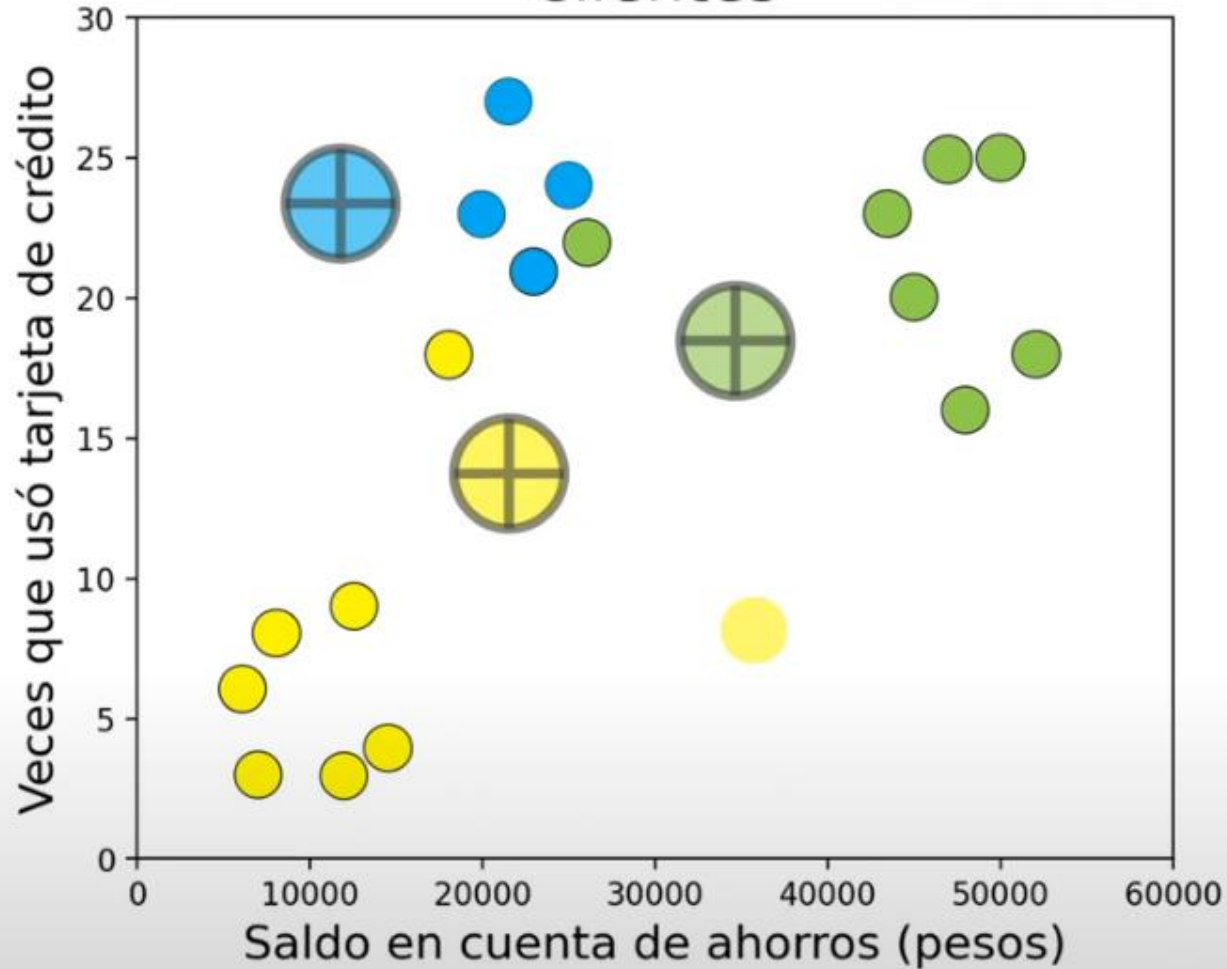
4



5

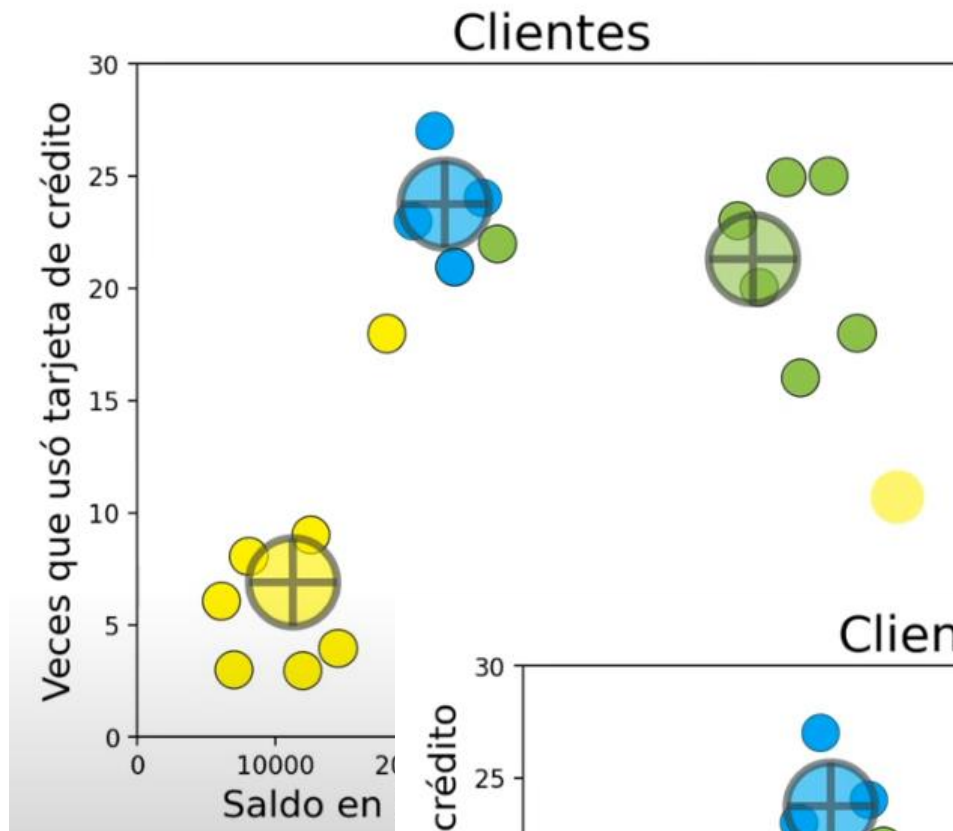


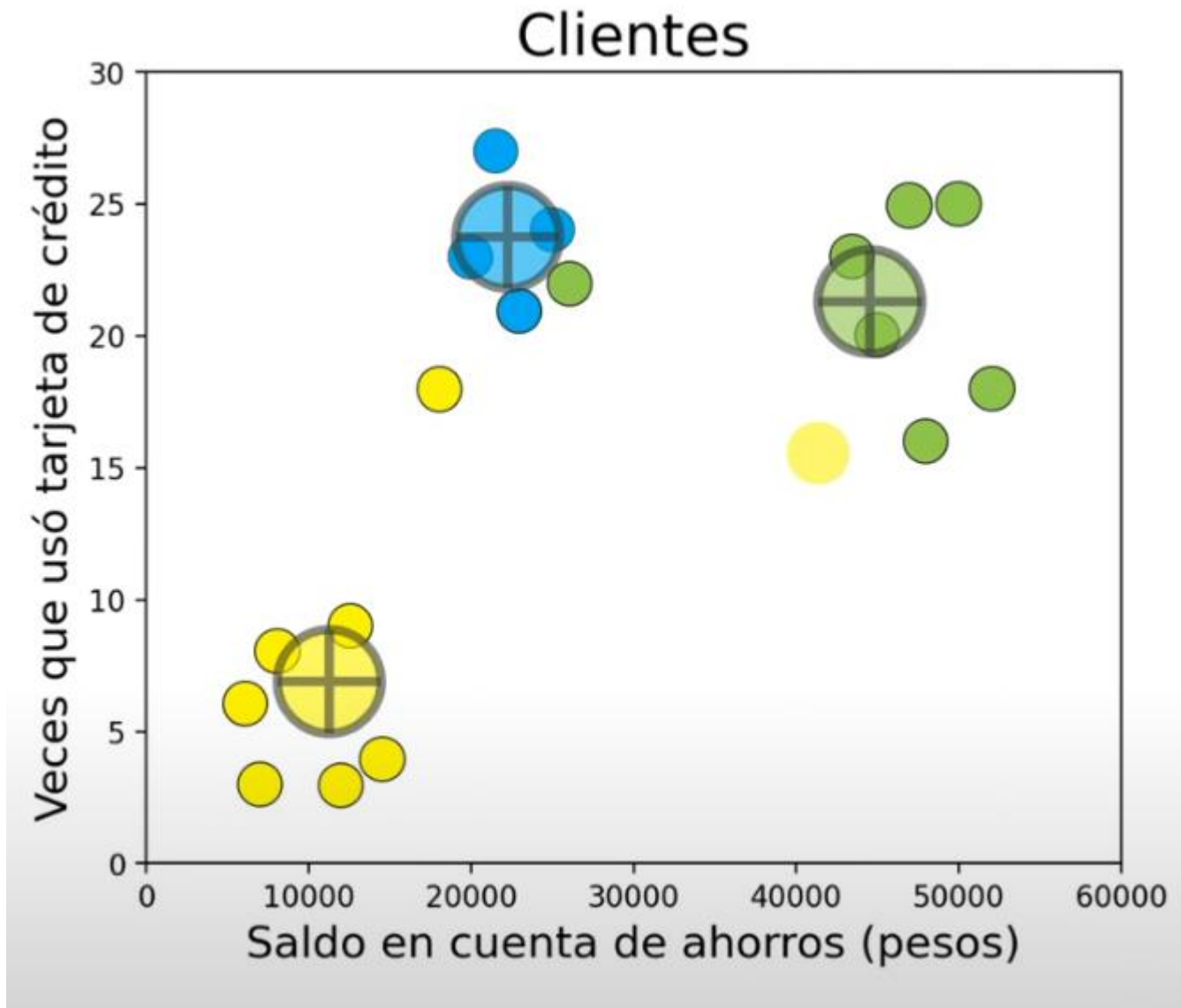
Cientes



Resultado de revisar cada punto:
Todos los puntos ya están asociados a un centroide. En otras palabras, ya asociamos a todos los clientes a su correspondiente clúster.

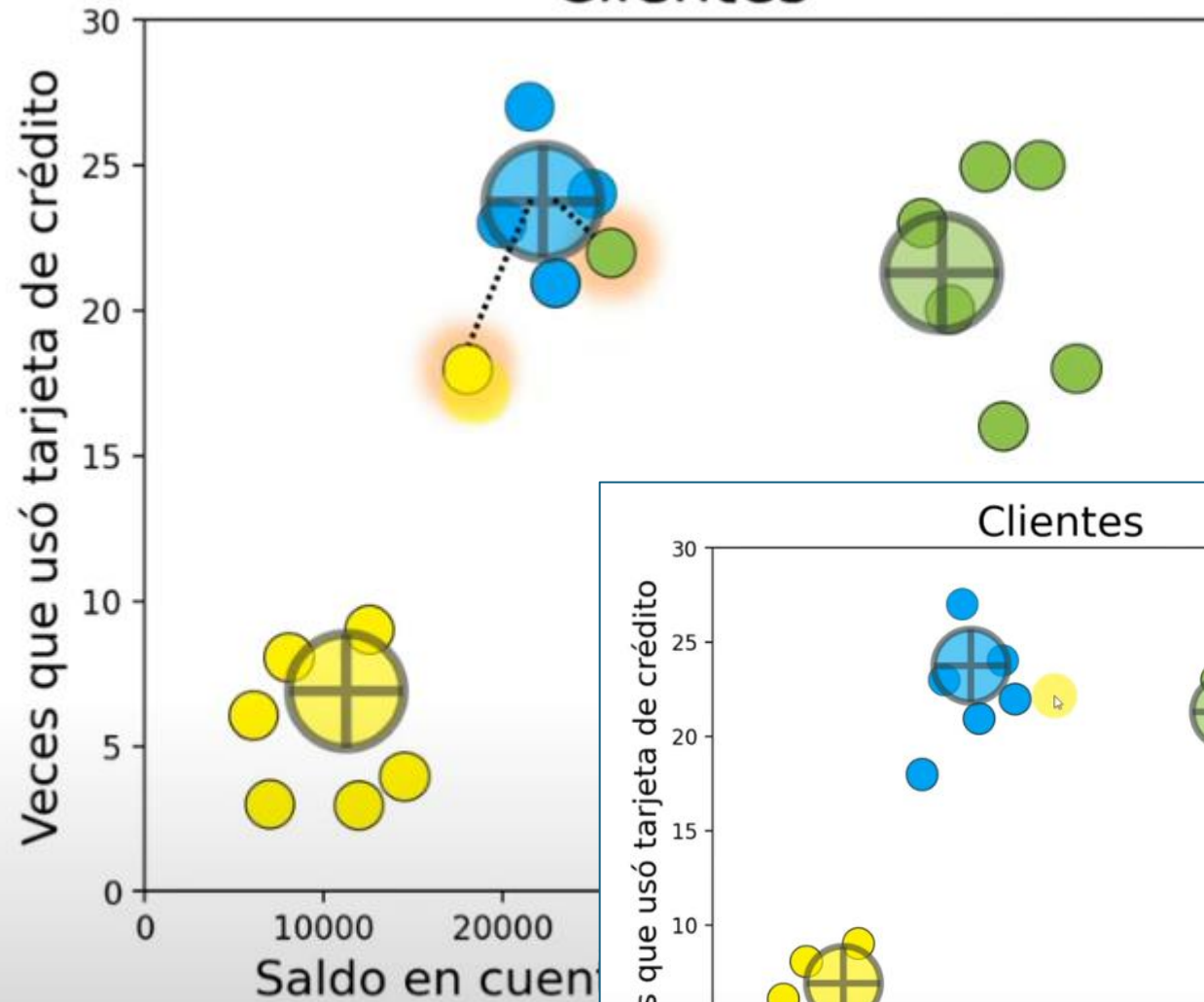
Paso 4: Recalculamos la posición de los centroides. Acercar el centroide al centro del clúster. Esto se realiza calculando el promedio de todas las ubicaciones de todos los puntos tanto del eje x como de y de cada cluster.





Repetimos el proceso inicial. Inicialmente, asumimos una posición aleatoria de los centroides. En esta 2 iteración, se cuenta con centroides donde su ubicación ya no es aleatoria sino guiada por la primera iteración.

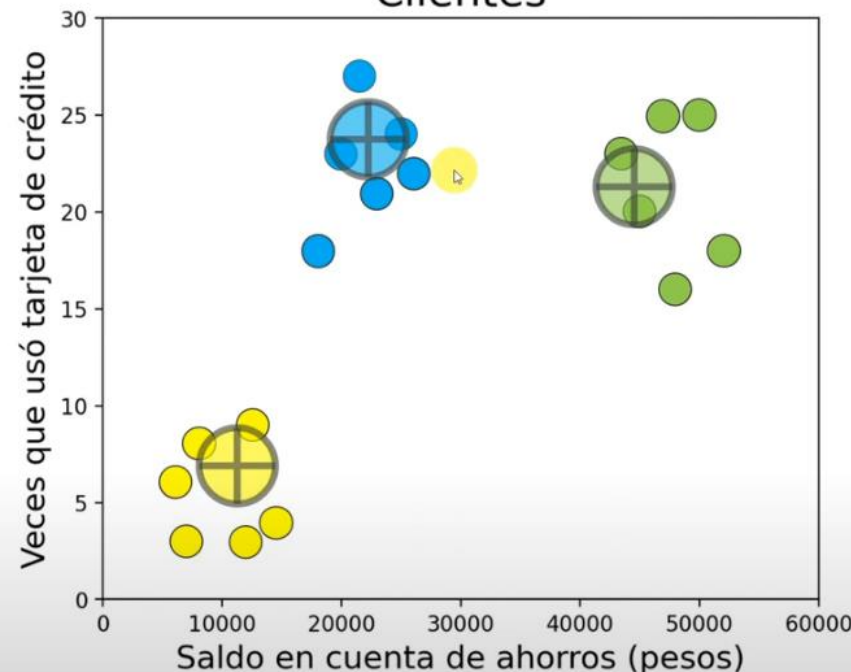
Cientes



Repetimos el proceso inicial.

Esto es, para todos los puntos calculamos la distancia euclidiana y lo asociamos al clúster cuya distancia sea menor.

Cientes



Nuevamente, ajustamos la posición de los centroides.

Luego repetimos el proceso y cuando vemos que la posición de los centroides no cambia está marcando el final del algoritmo.

K-means con Scikit-Learn

Revisar uso de librerías

K-means (K-medias)

Segmentación de clientes

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans

clientes = pd.DataFrame({"saldo" : [50000, 45000, 48000, 43500, 47000, 52000,
                                   20000, 26000, 25000, 23000, 21400, 18000,
                                   8000, 12000, 6000, 14500, 12600, 7000],
                        "transacciones": [25, 20, 16, 23, 25, 18,
                                           23, 22, 24, 21, 27, 18,
                                           8, 3, 6, 4, 9, 3]})
```

MinMaxScaler se ocupa porque el algoritmo de kmeans es susceptible o afectado por diferentes escalas dentro de nuestros datos

Escalas distintas

Evitar sesgos de las diferentes escalas por ende hay que convertir los valores de saldos entre 0 y 1 conservando la misma información

Consultamos el dataframe

clientes

	saldo	transacciones
0	0.956522	0.916667
1	0.847826	0.708333
2	0.913043	0.541667
3	0.815217	0.833333
4	0.891304	0.916667
5	1.000000	0.625000
6	0.304348	0.833333
7	0.434783	0.791667
8	0.413043	0.875000
9	0.369565	0.750000
10	0.334783	1.000000
11	0.260870	0.625000
12	0.043478	0.208333
13	0.130435	0.000000
14	0.000000	0.125000
15	0.184783	0.041667

Podemos observar que tenemos los valores entre 0 y 1

Creamos el modelo

```
kmeans = KMeans(n_clusters=3).fit(clientes.values)
```

El resultado de cada uno de los clientes esta almacenado en el atributo labels_

```
kmeans.labels_
```

array([2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])

Se agrega una nueva columna al dataframe indicando el cluster al cual pertenecen ..

```
clientes["cluster"] = kmeans.labels_
```

clientes

	saldo	transacciones	cluster
0	0.956522	0.916667	0
1	0.847826	0.708333	0
2	0.913043	0.541667	0
3	0.815217	0.833333	0
4	0.891304	0.916667	0
5	1.000000	0.625000	0
6	0.304348	0.833333	2
7	0.434783	0.791667	2
8	0.413043	0.875000	2
9	0.369565	0.750000	2
10	0.334783	1.000000	2
11	0.260870	0.625000	2
12	0.043478	0.208333	1
13	0.130435	0.000000	1

Imprimir los centroides ...y la métrica inercia

```
print(kmeans.cluster_centers_, kmeans.inertia_)
```

```
[[0.35289855 0.8125      ]  
 [0.08731884 0.10416667]  
 [0.90398551 0.75694444]] 0.3334968800322061
```

Inercia nos dice que tan pegados están los clientes de los centroides. Entre más pegados estén estas identificando mejor ese clúster.

```
# Instrucciones para graficar los clusters
```

```
plt.figure(figsize=(6, 5), dpi=100)
```

```
colores = ["red", "blue", "orange", "black", "purple", "pink", "brown"]
```

```
for cluster in range(kmeans.n_clusters):
```

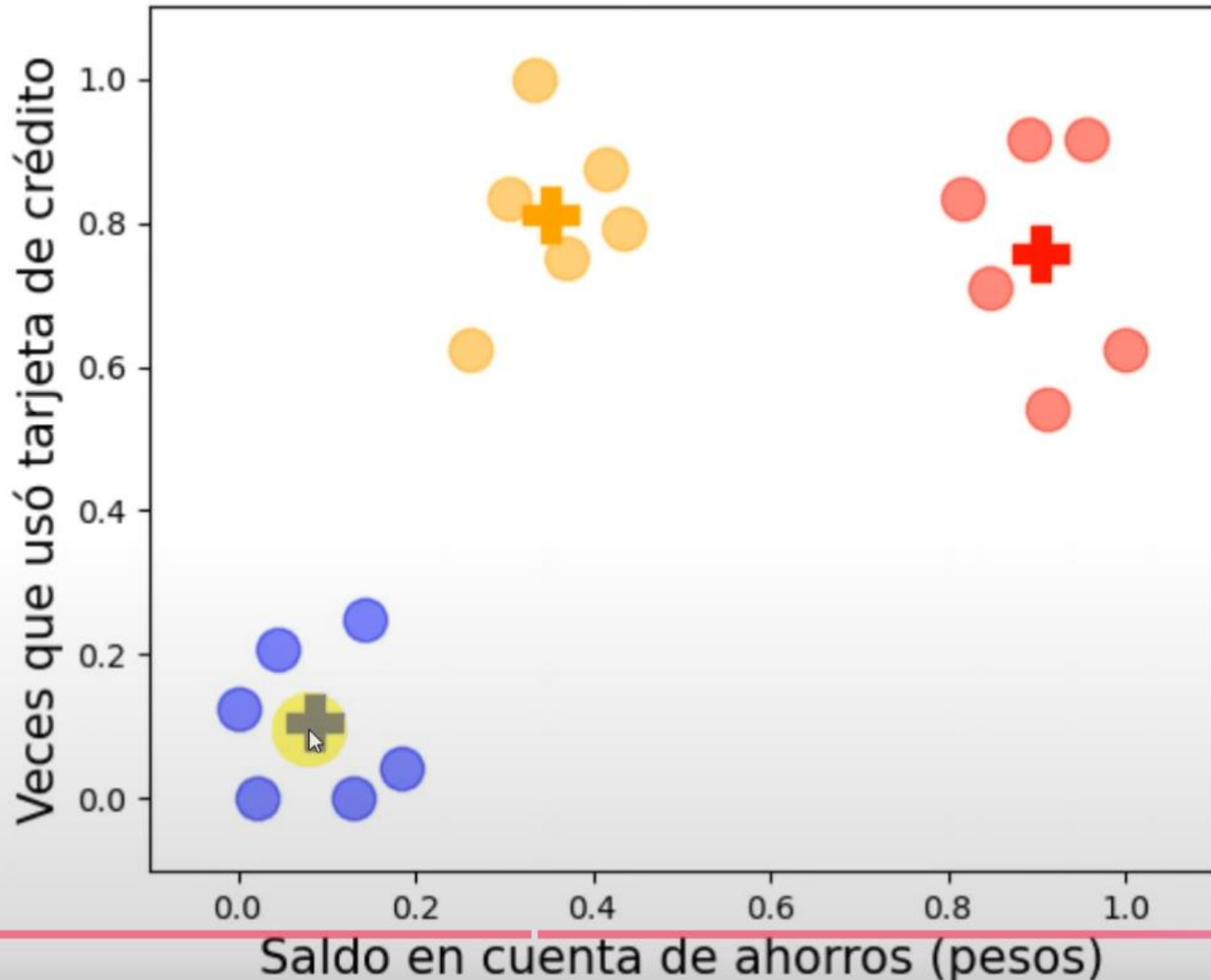
```
    plt.scatter(clientes[clientes["cluster"] == cluster]["saldo"],  
                clientes[clientes["cluster"] == cluster]["transacciones"],  
                marker="o", s=180, color=colores[cluster], alpha=0.5)
```

```
    plt.scatter(kmeans.cluster_centers_[cluster][0],  
                kmeans.cluster_centers_[cluster][1],  
                marker="P", s=280, color=colores[cluster])
```

```
plt.title("Clientes", fontsize=20)
plt.xlabel("Saldo en cuenta de ahorros (pesos)", fontsize=15)
plt.ylabel("Veces que usó tarjeta de crédito", fontsize=15)
plt.text(1.15, 0.2, "K = %i" % kmeans.n_clusters, fontsize=25)
plt.text(1.15, 0, "Inercia = %0.2f" % kmeans.inertia_, fontsize=25)
plt.xlim(-0.1, 1.1)
plt.ylim(-0.1, 1.1)
plt.show()

del clientes["cluster"]
```

Cientes

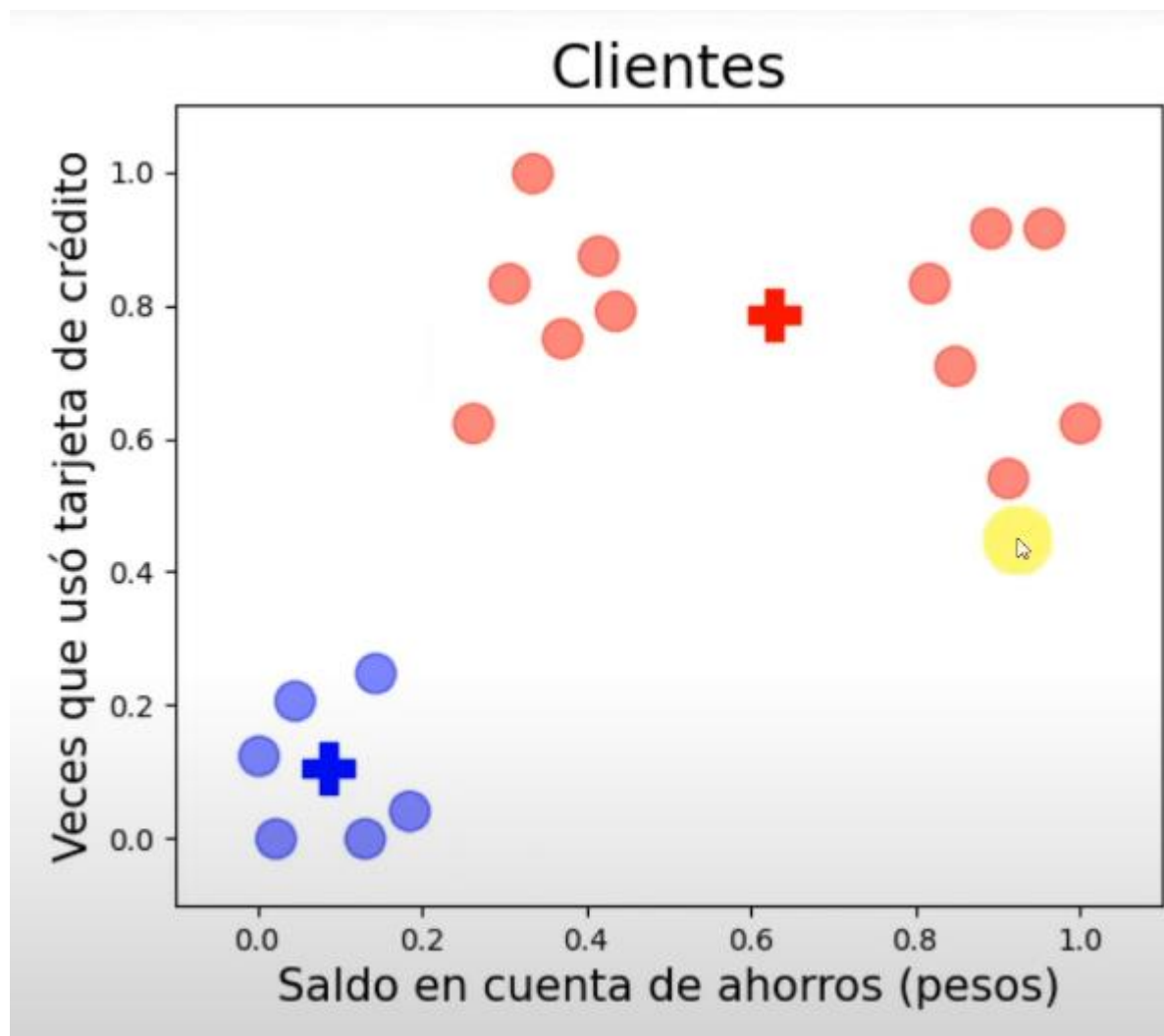


$$K = 3$$

$$\text{Inercia} = 0.33$$

Inercia nos dice que tan pegados están los clientes de los centroides. Entre más pegados estén estas identificando mejor ese clúster.

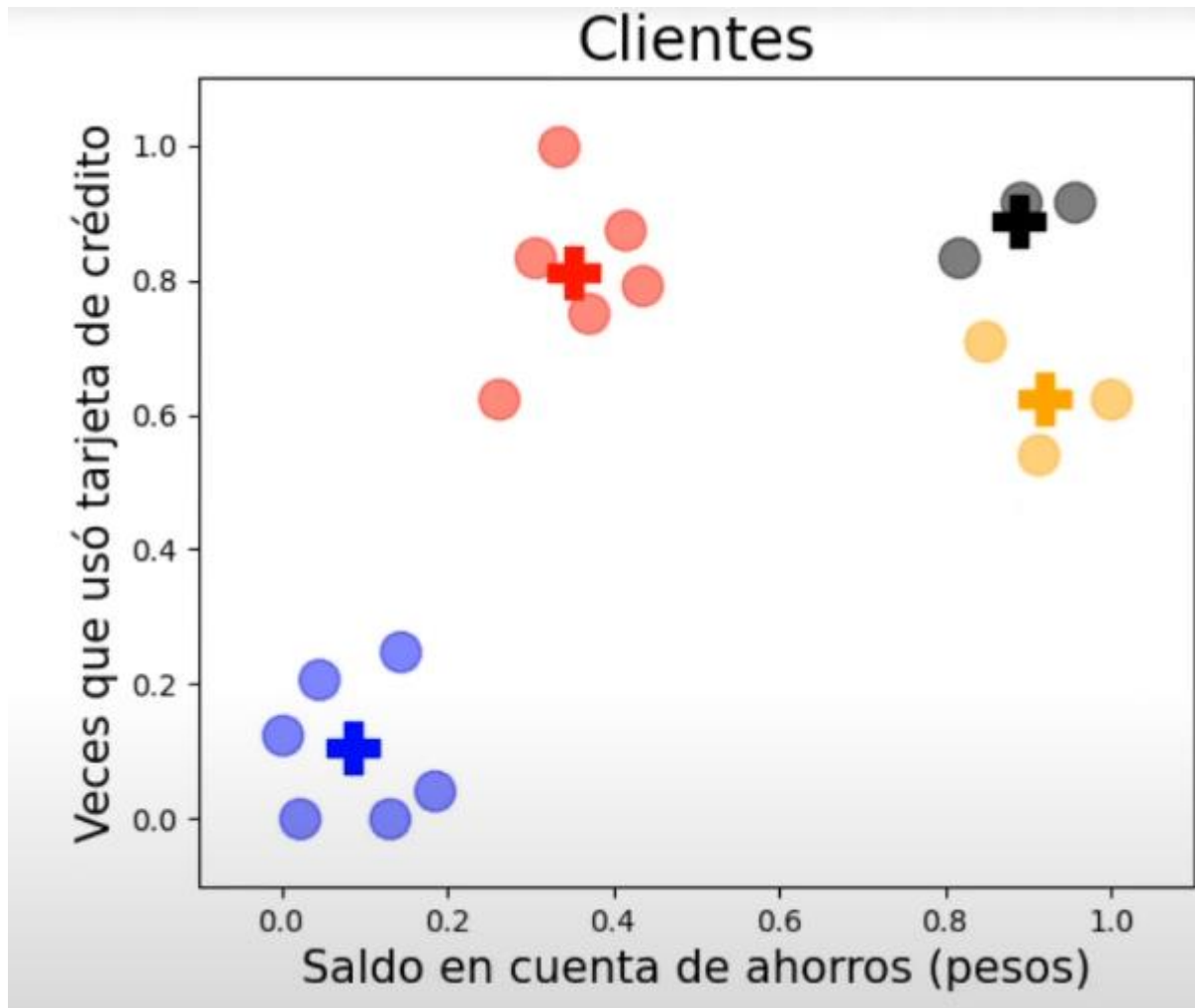
La inercia se calcula como la suma de todas las distancias al cuadrado de cada uno de los puntos hacia el centroide y las pondera por el número de clientes que se tienen para un clúster.



$K = 2$

Inercia = 1.25

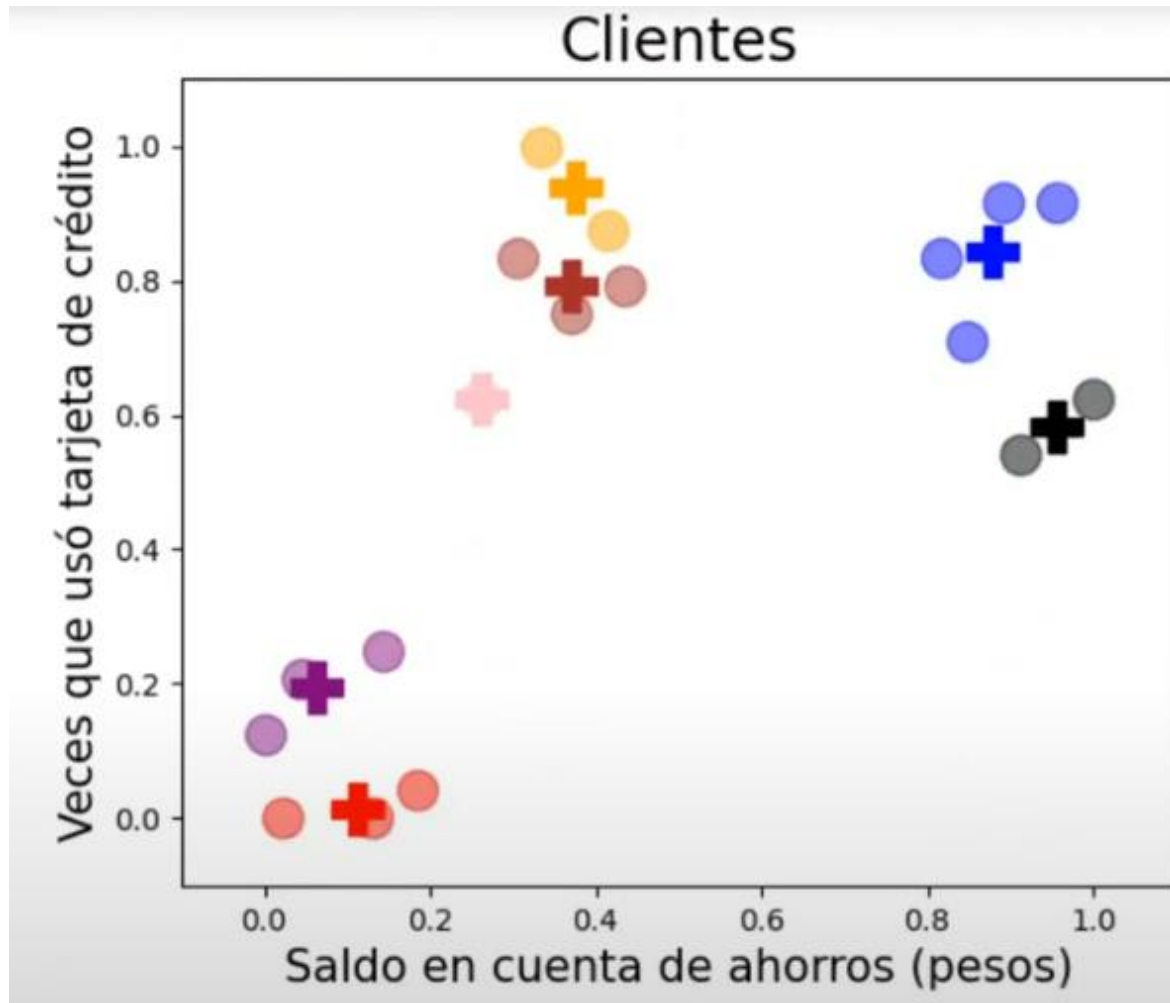
Aumento la inercia



$K = 4$

Inercia = 0.23

Aumentamos el K la inercia baja ...
Estamos caracterizando de mejor manera los cluster.



¿Qué paso con los datos?

Un solo cluster rosado de un solo punto ...¿esto dice algo? ...No? ...esto provoca que la inercia sea muy baja... pues Sí?

Recordar que estas métricas siempre deben estar asociadas al negocio al contexto de los datos que nosotros estamos analizando....No podemos dejarnos ir por una simple métrica que esta disasociada enteramente del negocio

Se dice que un buen modelo de clusterizado es aquel que tiene un valor de k pequeño y un valor de inercia pequeño sin embargo aquí eso un intercambio ... ya que si pones un K mayor la inercia tenderá a bajar

Por lo tanto, revisemos como calcular K a través de un método bastante sencillo llamado el método del codo.

$$K = 7$$

$$\text{Inercia} = 0.10$$

La inercia se calcula como la suma de todas las distancias al cuadrado de cada uno de los puntos hacia el centroide y las pondera por el número de clientes que se tienen para un clúster.

Calcular el valor de K

Método del Codo

Cálculo del número de Clusters (K)

```
In [ ]: inercias = []
        for k in range(2, 10):
            kmeans = KMeans(n_clusters=k).fit(clientes.values)
            inercias.append(kmeans.inertia_)

        plt.figure(figsize=(6, 5), dpi=100)
        plt.scatter(range(2, 10), inercias, marker="o", s=180, color="purple")
        plt.xlabel("Número de Clusters", fontsize=25)
        plt.ylabel("Inercia", fontsize=25)
        plt.show()
```

Este punto queda en el codo si pensáramos que es un brazo. Además, es este punto que se asume que se balancea un pequeño de la inercia y un valor pequeño de clúster. Considerar que todo depende del contexto.

